PL/pgSQL profesional

Funciones y procedimientos almacenados con PostgreSQL 18

Guía práctica basada en el esquema **tienda**, con código ejecutable, resultados esperados y nomenclatura coherente de variables.

Cómo usar este documento

El índice de la página siguiente es interactivo: pulsa una lección para ir directamente a ella. Los marcadores del visor de PDF ofrecen la misma navegación jerárquica.

Índice navegable

PL/pgSQL profesional desde cero — PostgreSQL 18 . . . 6

1. Qué son funciones, procedimientos y PL/pgSQL . . . 6

2. Primera función: retorno escalar y parámetros de entrada . . . 6

Enunciado . . . 6

Parámetros posicionales, nombrados y valores por defecto . . . 7

3. SELECT INTO, FOUND, STRICT y control de flujo . . . 8

Enunciado . . . 8

4. Parámetros IN, OUT, INOUT y VARIADIC . . . 9

Enunciado: resumen de un pedido con parámetros OUT . . . 9

INOUT . . . 9

VARIADIC . . . 10

5. Devolver conjuntos: SETOF, RETURNS TABLE, RETURN NEXT y RETURN QUERY . . . 10

5.1 RETURNS TABLE + RETURN QUERY (la opción más legible) . . . 10

Enunciado . . . 10

5.2 RETURN NEXT para construir filas una a una . . . 11

5.3 RETURNS SETOF tipo_base . . . 11

5.4 RETURNS SETOF tabla y %ROWTYPE . . . 11

5.5 RETURNS SETOF record . . . 12

5.6 SETOF record + RETURN NEXT v_fila . . . 12

Enunciado . . . 12

5.7 SETOF record + varios OUT + RETURN NEXT . . . 13

6. RECORD frente a %ROWTYPE . . . 13

Enunciado: una misma variable record para dos formas de fila . . . 14

7. SQL dinámico seguro: EXECUTE, USING, %I, %L y RETURN QUERY EXECUTE . . . 14

Enunciado: filtrar y ordenar productos por una columna autorizada . . . 14

8. Cursores: explícitos e implícitos . . . 15

8.1 Cursor implícito sin OUT, con %ROWTYPE . . . 15

Enunciado . . . 15

8.2 Cursor implícito con OUT/RETURNS TABLE y record . . . 16

8.3 Cursor explícito ligado (bound cursor) sin OUT . . . 16

8.4 Cursor explícito con RETURNS TABLE y record . . . 16

8.5 Cursor explícito con RETURNS TABLE, sin record . . . 17

8.6 Devolver un cursor al llamador (refcursor con OUT) . . . 17

9. Procedimientos almacenados (CREATE PROCEDURE) . . . 18

Enunciado . . . 18

Procedimiento con INOUT . . . 18

10. Agregaciones, subconsultas y consultas anidadas . . . 19

Enunciado: total por cliente con subconsulta correlacionada . . . 19

11. Funciones de ventana . . . 20

Enunciado: clasificar empleados por salario dentro de su departamento . . . 20

12. JSON y JSONB en PostgreSQL 18 . . . 20

12.1 Pedido con líneas: json_agg + json_build_object . . . 20

12.2 jsonb_agg + jsonb_build_object y operadores JSONB . . . 21

12.3 to_json, to_jsonb y row_to_json . . . 22

12.4 json_serialize en PostgreSQL 18 . . . 22

12.5 Retornar JSONB desde PL/pgSQL . . . 22

13. Errores, validación y buenas prácticas . . . 22

Seguridad de rutinas . . . 23

14. Mapa de decisión rápido . . . 23

15. Consultas de comprobación final . . . 24

16. Anexo: comparativa rápida de cursores 24

17. Anexo: reglas de oro con SETOF record 25

0. Script inicial: base de datos, tablas y datos

Ejecuta este bloque una sola vez, antes de las lecciones. Crea el esquema `tienda`, las tablas y los datos de ejemplo.

```
CREATE SCHEMA tienda;
SET search_path TO tienda;

CREATE TABLE clientes (
    cliente_id SERIAL PRIMARY KEY,
    nombre TEXT NOT NULL,
    email TEXT UNIQUE NOT NULL,
    fecha_alta DATE NOT NULL DEFAULT CURRENT_DATE
);

CREATE TABLE productos (
    producto_id SERIAL PRIMARY KEY,
    nombre TEXT NOT NULL,
    precio NUMERIC(10,2) NOT NULL,
    stock INTEGER NOT NULL DEFAULT 0,
    activo BOOLEAN NOT NULL DEFAULT TRUE
);

CREATE TABLE pedidos (
    pedido_id SERIAL PRIMARY KEY,
    cliente_id INTEGER NOT NULL REFERENCES clientes(cliente_id),
    fecha_pedido TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    estado TEXT NOT NULL DEFAULT 'pendiente'
);

CREATE TABLE lineas_pedido (
    linea_id SERIAL PRIMARY KEY,
    pedido_id INTEGER NOT NULL REFERENCES pedidos(pedido_id),
    producto_id INTEGER NOT NULL REFERENCES productos(producto_id),
    cantidad INTEGER NOT NULL,
    precio_unit NUMERIC(10,2) NOT NULL
);

CREATE TABLE pagos (
    pago_id SERIAL PRIMARY KEY,
    pedido_id INTEGER NOT NULL REFERENCES pedidos(pedido_id),
    fecha_pago TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    importe NUMERIC(10,2) NOT NULL,
    metodo TEXT NOT NULL
);

CREATE TABLE empleados (
    id SERIAL PRIMARY KEY,
    nombre TEXT NOT NULL,
    salario NUMERIC(10,2) NOT NULL,
    departamento TEXT,
    fecha_contratacion DATE DEFAULT CURRENT_DATE
);

INSERT INTO empleados (nombre, salario, departamento) VALUES
    ('Ana Torres', 2800.00, 'Ventas'),
    ('Luis Martínez', 4500.00, 'IT'),
    ('Carla Gómez', 5200.00, 'IT'),
    ('Dmitry Komarov', 3100.00, 'Contabilidad'),
    ('James Smith', 7000.00, 'Dirección'),
    ('Mary Sexton', 3900.00, 'Marketing'),
    ('Agnes Miller', 6100.00, 'IT'),
    ('Chun Yen', 2950.00, 'Ventas');

INSERT INTO clientes (nombre, email) VALUES
    ('Juan Pérez', 'juan@example.com'),
    ('María López', 'maria@example.com');

INSERT INTO productos (nombre, precio, stock) VALUES
    ('Teclado mecánico', 79.90, 50),
    ('Ratón inalámbrico', 39.50, 100),
    ('Monitor 27"', 249.00, 20);

INSERT INTO pedidos (cliente_id, estado) VALUES
    (1, 'pendiente'),
    (2, 'pendiente');

INSERT INTO lineas_pedido (pedido_id, producto_id, cantidad, precio_unit) VALUES
    (1, 1, 1, 79.90),
    (1, 2, 2, 39.50),
    (2, 3, 1, 249.00);

INSERT INTO pagos (pedido_id, importe, metodo) VALUES
    (1, 158.90, 'tarjeta'),
    (2, 249.00, 'transferencia');
```

PL/pgSQL profesional desde cero — PostgreSQL 18

Este tutorial usa el esquema `tienda` y los datos que proporcionaste. Ejecuta primero tu script de creación e inserción y, antes de los ejemplos, establece el esquema:

```
SET search_path TO tienda, public;
```

> *Convención de este documento: toda variable PL/pgSQL y toda columna de salida se llama con el prefijo `v_`: `v_linea_id`, `v_pedido_id`, `v_total`, etc. Es una convención de lectura; PostgreSQL no la exige.*

1. Qué son funciones, procedimientos y PL/pgSQL

PL/pgSQL es el lenguaje procedural de PostgreSQL. Permite declarar variables, tomar decisiones, iterar, controlar excepciones y combinar SQL en objetos reutilizables.

Objeto	Se invoca con	Devuelve datos	Puede controlar transacciones
Función (`FUNCTION`)	`SELECT`, `PERFORM`	Sí: escalar, fila, tabla, cursor…	No `COMMIT`/`ROLLBACK` dentro de una función
Procedimiento (`PROCEDURE`)	`CALL`	Puede devolver valores mediante `OUT`/`INOUT`	Sí, cuando `CALL` se ejecuta en un contexto válido y no hay bloque de transacción envolvente

Una función debe tener un `RETURNS`; una función con parámetros `OUT` lo deduce. Un procedimiento no usa `RETURNS`.

Plantilla básica:

```
CREATE OR REPLACE FUNCTION tienda.nombre_funcion(
    p_valor integer
)
RETURNS integer
LANGUAGE plpgsql
AS $$
DECLARE
    v_resultado integer;
BEGIN
    v_resultado := p_valor * 2;
    RETURN v_resultado;
END;
$$;
```

Puntos importantes

- p_ identifica parámetros de entrada; v_, variables y salidas. Es una disciplina útil para evitar ambigüedades con nombres de columnas.
- `:=` asigna en PL/pgSQL. `SELECT ... INTO` copia el resultado de una consulta a una variable.
- Usa `CREATE OR REPLACE` para evolucionar el cuerpo. Cambiar la firma (tipos/número de parámetros) requiere normalmente `DROP FUNCTION` o crear otra sobrecarga.
- Especifica `tienda.` al crear objetos si no quieres depender de `search_path`.

2. Primera función: retorno escalar y parámetros de entrada

Enunciado

Crear una función que calcule el importe de una línea, redondeado a dos decimales.

```
CREATE OR REPLACE FUNCTION tienda.fn_importe_linea(
    p_cantidad integer,
    p_precio_unit numeric(10,2)
)
RETURNS numeric(10,2)
LANGUAGE plpgsql
IMMUTABLE
AS $$
DECLARE
    v_importe numeric(10,2);
BEGIN
    IF p_cantidad <= 0 OR p_precio_unit < 0 THEN
        RAISE EXCEPTION 'Cantidad (%) y precio (%) deben ser válidos',
            p_cantidad, p_precio_unit
            USING ERRCODE = '22023';
    END IF;

    v_importe := round(p_cantidad * p_precio_unit, 2);
    RETURN v_importe;
END;
$$;

SELECT tienda.fn_importe_linea(2, 39.50);
```

Resultado esperado

fn_importe_linea
79.00

`IMMUTABLE` significa que, para iguales entradas, el resultado no cambia y no depende de tablas ni de la hora. No declares una función `IMMUTABLE` si consulta datos o usa `CURRENT_TIMESTAMP`; en esos casos usa el valor por defecto `VOLATILE` (o, si aplica, `STABLE`).

Parámetros posicionales, nombrados y valores por defecto

```
CREATE OR REPLACE FUNCTION tienda.fn_precio_con_iva(
    p_precio numeric(10,2),
    p_porcentaje_iva numeric(5,2) DEFAULT 21
)
RETURNS numeric(10,2)
LANGUAGE plpgsql
IMMUTABLE
AS $$
BEGIN
    RETURN round(p_precio * (1 + p_porcentaje_iva / 100), 2);
END;
$$;

-- Posicional: el orden determina cada argumento.
SELECT tienda.fn_precio_con_iva(100.00, 10.00) AS v_precio;
-- Resultado: 110.00

-- Nombrado: más legible; el orden no importa.
SELECT tienda.fn_precio_con_iva(p_porcentaje_iva => 4, p_precio => 100.00) AS v_precio;
-- Resultado: 104.00

-- Se usa el valor predeterminado.
SELECT tienda.fn_precio_con_iva(100.00) AS v_precio;
-- Resultado: 121.00
```

Dentro del cuerpo también se pueden referenciar los argumentos por posición: $1, $2, etc. Es especialmente cómodo si el parámetro no tiene nombre.

```
CREATE OR REPLACE FUNCTION tienda.fn_margen(
    numeric(10,2), numeric(10,2)
)
RETURNS numeric(10,2)
LANGUAGE plpgsql
IMMUTABLE
AS $$
BEGIN
    RETURN round($1 - $2, 2); -- $1 = venta, $2 = coste
END;
```

```
$$;

SELECT tienda.fn_margen(150.00, 90.00) AS v_margen;
-- Resultado: 60.00
```

No confundas `$1` de una función con `$1` de `EXECUTE`: en SQL dinámico, `$1` es un marcador que recibe valores mediante `USING`.

3. `SELECT INTO`, `FOUND`, `STRICT` y control de flujo

Enunciado

Consultar el stock de un producto y fallar si no existe.

```
CREATE OR REPLACE FUNCTION tienda.fn_stock_producto(
    p_producto_id integer
)
RETURNS integer
LANGUAGE plpgsql
STABLE
AS $$
DECLARE
    v_stock integer;
BEGIN
    SELECT p.stock
      INTO STRICT v_stock
      FROM tienda.productos AS p
     WHERE p.producto_id = p_producto_id;

    RETURN v_stock;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RAISE EXCEPTION 'No existe el producto %', p_producto_id
            USING ERRCODE = 'P0002';
    WHEN TOO_MANY_ROWS THEN
        RAISE EXCEPTION 'La búsqueda de producto % no es única', p_producto_id;
END;
$$;

SELECT tienda.fn_stock_producto(1) AS v_stock;
```

Resultado esperado: `50`.

- `INTO STRICT` exige exactamente una fila: lanza `NO_DATA_FOUND` o `TOO_MANY_ROWS`.
- Sin `STRICT`, cero filas deja la variable en `NULL`; varias filas guardan la primera (no es una práctica aconsejable si se espera unicidad).
- Después de `SELECT INTO`, `FOUND` indica si apareció al menos una fila.
- Si solo deseas ejecutar una función y descartar su valor, usa `PERFORM`, no `SELECT`: `PERFORM tienda.fn_cambiar_estado(1, 'pagado');`.

```
CREATE OR REPLACE FUNCTION tienda.fn_existe_producto(
    p_producto_id integer
)
RETURNS boolean
LANGUAGE plpgsql
STABLE
AS $$
DECLARE
    v_nombre text;
BEGIN
    SELECT p.nombre INTO v_nombre
    FROM tienda.productos AS p
    WHERE p.producto_id = p_producto_id;

    RETURN FOUND;
END;
$$;

SELECT tienda.fn_existe_producto(99) AS v_existe;
```

```
-- Resultado: false
```

Bloques de control más usados:

```
IF condicion THEN
    -- ...
ELSIF otra_condicion THEN
    -- ...
ELSE
    -- ...
END IF;

CASE p_estado
    WHEN 'pendiente' THEN ...
    WHEN 'pagado' THEN ...
    ELSE RAISE EXCEPTION 'Estado no reconocido: %', p_estado;
END CASE;

FOR v_indice IN 1..5 LOOP
    -- EXIT WHEN condicion;
    -- CONTINUE WHEN condicion;
END LOOP;
```

4. Parámetros `IN`, `OUT`, `INOUT` y `VARIADIC`

`IN` es el modo predeterminado. `OUT` es una variable local inicializada a `NULL` y forma parte de la salida. `INOUT` entra con un valor y sale, posiblemente cambiado. Una función con varios `OUT` devuelve conceptualmente una fila `record`.

Enunciado: resumen de un pedido con parámetros `OUT`

```
CREATE OR REPLACE FUNCTION tienda.fn_resumen_pedido_out(
    IN p_pedido_id integer,
    OUT v_pedido_id integer,
    OUT v_cliente_nombre text,
    OUT v_total numeric(10,2),
    OUT v_estado text
)
LANGUAGE plpgsql
STABLE
AS $$
BEGIN
    SELECT pe.pedido_id,
           c.nombre,
           COALESCE(SUM(lp.cantidad * lp.precio_unit), 0),
           pe.estado
      INTO STRICT v_pedido_id, v_cliente_nombre, v_total, v_estado
      FROM tienda.pedidos AS pe
      JOIN tienda.clientes AS c ON c.cliente_id = pe.cliente_id
      LEFT JOIN tienda.lineas_pedido AS lp ON lp.pedido_id = pe.pedido_id
     WHERE pe.pedido_id = p_pedido_id
     GROUP BY pe.pedido_id, c.nombre, pe.estado;
END;
$$;

SELECT * FROM tienda.fn_resumen_pedido_out(1);
```

Resultado esperado

v_pedido_id	v_cliente_nombre	v_total	v_estado
1	Juan Pérez	158.90	pendiente

`INOUT`

```
CREATE OR REPLACE FUNCTION tienda.fn_aplicar_descuento(
    INOUT v_importe numeric(10,2),
    IN p_porcentaje numeric(5,2)
)
LANGUAGE plpgsql
IMMUTABLE
```

```
AS $$
BEGIN
    IF p_porcentaje NOT BETWEEN 0 AND 100 THEN
        RAISE EXCEPTION 'Porcentaje inválido: %', p_porcentaje;
    END IF;
    v_importe := round(v_importe * (1 - p_porcentaje / 100), 2);
END;
$$;

SELECT tienda.fn_aplicar_descuento(200.00, 15.00) AS v_importe_final;
-- Resultado: 170.00
```

VARIADIC

```
CREATE OR REPLACE FUNCTION tienda.fn_suma_importes(
    VARIADIC p_importes numeric[]
)
RETURNS numeric
LANGUAGE sql
IMMUTABLE
AS $$
    SELECT COALESCE(SUM(v_importe), 0)
    FROM unnest(p_importes) AS u(v_importe);
$$;

SELECT tienda.fn_suma_importes(10.00, 20.50, 3.25) AS v_total;
-- Resultado: 33.75
```

5. Devolver conjuntos: SETOF, RETURNS TABLE, RETURN NEXT y RETURN QUERY

5.1 RETURNS TABLE + RETURN QUERY (la opción más legible)

Enunciado

Listar productos activos cuyo stock sea igual o superior a un mínimo.

```
CREATE OR REPLACE FUNCTION tienda.fn_productos_por_stock(
    p_stock_minimo integer DEFAULT 0
)
RETURNS TABLE(
    v_producto_id integer,
    v_nombre text,
    v_precio numeric(10,2),
    v_stock integer
)
LANGUAGE plpgsql
STABLE
AS $$
BEGIN
    RETURN QUERY
    SELECT p.producto_id, p.nombre, p.precio, p.stock
    FROM tienda.productos AS p
    WHERE p.activo
      AND p.stock >= p_stock_minimo
    ORDER BY p.producto_id;
END;
$$;

SELECT * FROM tienda.fn_productos_por_stock(50);
```

Resultado esperado

v_producto_id	v_nombre	v_precio	v_stock
1	Teclado mecánico	79.90	50
2	Ratón inalámbrico	39.50	100

En `RETURNS TABLE`, las columnas declaradas son variables `OUT`; por eso también llevan `v_`. No se pueden mezclar `OUT`/`INOUT` explícitos con `RETURNS TABLE`.

5.2 `RETURN NEXT` para construir filas una a una

```
CREATE OR REPLACE FUNCTION tienda.fn_numeros_pedido(
    p_hasta integer
)
RETURNS TABLE(v_numero integer, v_cuadrado integer)
LANGUAGE plpgsql
IMMUTABLE
AS $$
DECLARE
    v_indice integer;
BEGIN
    FOR v_indice IN 1..p_hasta LOOP
        v_numero := v_indice;
        v_cuadrado := v_indice * v_indice;
        RETURN NEXT; -- guarda los OUT actuales
    END LOOP;
    RETURN;
END;
$$;

SELECT * FROM tienda.fn_numeros_pedido(3);
-- (1,1), (2,4), (3,9)
```

`RETURN QUERY` agrega todas las filas de una consulta; `RETURN NEXT` agrega una fila. Se pueden combinar y el resultado conserva ese orden de adición. Ninguno termina por sí solo la función: el `RETURN` final sin expresión (o llegar al final) la termina.

5.3 `RETURNS SETOF tipo_base`

```
CREATE OR REPLACE FUNCTION tienda.fn_ids_productos_activos()
RETURNS SETOF integer
LANGUAGE plpgsql
STABLE
AS $$
BEGIN
    RETURN QUERY
    SELECT p.producto_id
    FROM tienda.productos AS p
    WHERE p.activo
    ORDER BY p.producto_id;
END;
$$;

SELECT * FROM tienda.fn_ids_productos_activos();
-- 1, 2, 3
```

5.4 `RETURNS SETOF tabla` y `%ROWTYPE`

```
CREATE OR REPLACE FUNCTION tienda.fn_productos_activos()
RETURNS SETOF tienda.productos
LANGUAGE plpgsql
STABLE
AS $$
DECLARE
    v_producto tienda.productos%ROWTYPE;
BEGIN
    FOR v_producto IN
        SELECT * FROM tienda.productos WHERE activo ORDER BY producto_id
    LOOP
        RETURN NEXT v_producto;
    END LOOP;
    RETURN;
END;
$$;

SELECT producto_id, nombre, stock FROM tienda.fn_productos_activos();
```

`%ROWTYPE` es una fila con la estructura conocida de una tabla, vista o cursor. PostgreSQL valida sus campos en la compilación/ejecución. Es preferible cuando la forma de la fila es estable y conocida.

5.5 `RETURNS SETOF record`

Úsalo cuando la forma de salida no se puede declarar de forma más precisa. El llamador **debe** proporcionar una definición de columnas.

```
CREATE OR REPLACE FUNCTION tienda.fn_productos_record(
    p_stock_minimo integer
)
RETURNS SETOF record
LANGUAGE plpgsql
STABLE
AS $$
BEGIN
    RETURN QUERY
    SELECT p.producto_id, p.nombre, p.stock
    FROM tienda.productos AS p
    WHERE p.stock >= p_stock_minimo
    ORDER BY p.producto_id;
END;
$$;

SELECT *
FROM tienda.fn_productos_record(50)
     AS r(v_producto_id integer, v_nombre text, v_stock integer);
```

`record` es una variable o tipo de salida de estructura **no fijada** hasta que recibe una fila. Es flexible; a cambio, se detectan más errores en ejecución y el cliente necesita saber las columnas. Prefiere `RETURNS TABLE` para una API clara.

5.6 `SETOF record` + `RETURN NEXT v_fila`

Enunciado

Devolver los productos con stock mínimo construyendo el conjunto fila a fila desde una variable `record`.

```
CREATE OR REPLACE FUNCTION tienda.fn_productos_record_return_next(
    p_stock_minimo integer
)
RETURNS SETOF record
LANGUAGE plpgsql
STABLE
AS $$
DECLARE
    v_fila record;
BEGIN
    FOR v_fila IN
        SELECT p.producto_id, p.nombre, p.stock
        FROM tienda.productos AS p
        WHERE p.stock >= p_stock_minimo
        ORDER BY p.producto_id
    LOOP
        RETURN NEXT v_fila; -- añade el contenido actual de la variable record
    END LOOP;
    RETURN;
END;
$$;

SELECT *
FROM tienda.fn_productos_record_return_next(50)
     AS r(v_producto_id integer, v_nombre text, v_stock integer);
```

Resultado esperado

v_producto_id	v_nombre	v_stock
1	Teclado mecánico	50

v_producto_id	v_nombre	v_stock
2	Ratón inalámbrico	100

El tipo de `v_fila` se concreta en cada vuelta por la consulta del `FOR`. Al no existir una estructura declarada en la firma, quien llama debe aportar `AS r(...)` con nombres y tipos compatibles.

5.7 `SETOF record` + varios `OUT` + `RETURN NEXT`

Con múltiples parámetros `OUT`, esos parámetros son el **conjunto de variables de salida** de la función. En este caso se usa `RETURN NEXT` sin expresión: PostgreSQL guarda como una fila los valores actuales de `v_linea_id`, `v_producto_id`, `v_cantidad` y `v_subtotal`.

```
CREATE OR REPLACE FUNCTION tienda.fn_lineas_out_setof_record(
    IN p_pedido_id integer,
    OUT v_linea_id integer,
    OUT v_producto_id integer,
    OUT v_cantidad integer,
    OUT v_subtotal numeric(10,2)
)
RETURNS SETOF record
LANGUAGE plpgsql
STABLE
AS $$
DECLARE
    v_fila record;
BEGIN
    FOR v_fila IN
        SELECT lp.linea_id,
               lp.producto_id,
               lp.cantidad,
               round(lp.cantidad * lp.precio_unit, 2) AS subtotal
        FROM tienda.lineas_pedido AS lp
        WHERE lp.pedido_id = p_pedido_id
        ORDER BY lp.linea_id
    LOOP
        v_linea_id := v_fila.linea_id;
        v_producto_id := v_fila.producto_id;
        v_cantidad := v_fila.cantidad;
        v_subtotal := v_fila.subtotal;
        RETURN NEXT; -- añade los OUT actuales como una nueva fila
    END LOOP;
    RETURN;
END;
$$;

SELECT * FROM tienda.fn_lineas_out_setof_record(1);
```

Resultado esperado

v_linea_id	v_producto_id	v_cantidad	v_subtotal
1	1	1	79.90
2	2	2	79.00

Regla práctica: `RETURN NEXT v_fila` se usa cuando devuelves una variable escalar, compuesta o `record`; `RETURN NEXT` sin argumento se usa en una función que tiene parámetros `OUT`/`RETURNS TABLE`, porque sus variables de salida ya representan la fila.

6. `RECORD` frente a `%ROWTYPE`

Aspecto	`v_producto tienda.productos%ROWTYPE`	`v_fila record`
Estructura	La de `tienda.productos`	Se determina al asignar
Uso ideal	`SELECT *` de una tabla/cursor conocido	Consultas distintas, SQL dinámico, joins

Aspecto	`v_producto tienda.productos%ROWTYPE`	`v_fila record`
Campos	Conocidos por la tabla	Solo tras asignar una fila
Flexibilidad	Menor	Mayor

Enunciado: una misma variable `record` para dos formas de fila

```
CREATE OR REPLACE FUNCTION tienda.fn_demostracion_record(
    p_incluir_stock boolean
)
RETURNS text
LANGUAGE plpgsql
STABLE
AS $$
DECLARE
    v_fila record;
BEGIN
    IF p_incluir_stock THEN
        SELECT p.nombre, p.stock INTO v_fila
        FROM tienda.productos AS p WHERE p.producto_id = 1;
        RETURN format('%s: %s unidades', v_fila.nombre, v_fila.stock);
    ELSE
        SELECT p.nombre, p.precio INTO v_fila
        FROM tienda.productos AS p WHERE p.producto_id = 1;
        RETURN format('%s: %s €', v_fila.nombre, v_fila.precio);
    END IF;
END;
$$;
```

7. SQL dinámico seguro: `EXECUTE`, `USING`, `%I`, `%L` y `RETURN QUERY EXECUTE`

Las consultas SQL estáticas se planifican y pueden reutilizar su plan. Usa SQL dinámico solo cuando cambian identificadores (tabla, columna, orden). **Nunca** concatenes valores de usuario: usa `USING`.

Enunciado: filtrar y ordenar productos por una columna autorizada

```
CREATE OR REPLACE FUNCTION tienda.fn_buscar_productos_dinamico(
    p_stock_minimo integer,
    p_ordenar_por text DEFAULT 'nombre'
)
RETURNS TABLE(
    v_producto_id integer,
    v_nombre text,
    v_precio numeric(10,2),
    v_stock integer
)
LANGUAGE plpgsql
STABLE
AS $$
DECLARE
    v_columna_orden text;
BEGIN
    v_columna_orden := CASE p_ordenar_por
        WHEN 'nombre' THEN 'nombre'
        WHEN 'precio' THEN 'precio'
        WHEN 'stock' THEN 'stock'
        ELSE RAISE EXCEPTION 'Orden no permitido: %', p_ordenar_por;
    END;

    RETURN QUERY EXECUTE format(
        'SELECT producto_id, nombre, precio, stock
           FROM tienda.productos
          WHERE activo AND stock >= $1
          ORDER BY %I, producto_id',
        v_columna_orden
    ) USING p_stock_minimo;
END;
$$;
```

```
SELECT * FROM tienda.fn_buscar_productos_dinamico(20, 'precio');
```

- `%I` en `format()` escapa un **identificador** (tabla/columna).
- `%L` escapa un **literal**, pero para valores se prefiere `$1 ... USING`.
- En `EXECUTE`, `$1` y `$2` son marcadores de la sentencia dinámica, no de la firma de la función.
- `RETURN QUERY EXECUTE ... USING ...` devuelve directamente el conjunto del SQL dinámico.

Ejemplo de `EXECUTE ... INTO` para una sola fila:

```
CREATE OR REPLACE FUNCTION tienda.fn_contar_filas(
    p_tabla text
)
RETURNS bigint
LANGUAGE plpgsql
STABLE
AS $$
DECLARE
    v_total bigint;
BEGIN
    IF p_tabla NOT IN ('clientes', 'productos', 'pedidos') THEN
        RAISE EXCEPTION 'Tabla no autorizada: %', p_tabla;
    END IF;
    EXECUTE format('SELECT count(*) FROM tienda.%I', p_tabla)
       INTO v_total;
    RETURN v_total;
END;
$$;
```

8. Cursores: explícitos e implícitos

Un cursor permite leer filas gradualmente. En PL/pgSQL, los bucles `FOR ... IN SELECT` usan internamente un cursor: normalmente son la mejor opción. Usa cursores explícitos para `FETCH`, desplazamiento, devolver un `refcursor` o `WHERE CURRENT OF`.

8.1 Cursor implícito sin `OUT`, con `%ROWTYPE`

Enunciado

Calcular el valor de inventario recorriendo productos activos.

```
CREATE OR REPLACE FUNCTION tienda.fn_valor_inventario_implicito()
RETURNS numeric(12,2)
LANGUAGE plpgsql
STABLE
AS $$
DECLARE
    v_producto tienda.productos%ROWTYPE;
    v_total numeric(12,2) := 0;
BEGIN
    FOR v_producto IN
        SELECT * FROM tienda.productos WHERE activo
    LOOP
        v_total := v_total + v_producto.precio * v_producto.stock;
    END LOOP;
    RETURN v_total;
END;
$$;

SELECT tienda.fn_valor_inventario_implicito() AS v_valor_inventario;
-- Resultado: 14420.00
```

El cursor es implícito: no se declara ni se abre/cierra manualmente.

8.2 Cursor implícito con `OUT`/`RETURNS TABLE` y `record`

```
CREATE OR REPLACE FUNCTION tienda.fn_lineas_pedido_implicito_record(
    p_pedido_id integer
)
RETURNS TABLE(
    v_linea_id integer,
    v_producto text,
    v_cantidad integer,
    v_subtotal numeric(10,2)
)
LANGUAGE plpgsql
STABLE
AS $$
DECLARE
    v_fila record;
BEGIN
    FOR v_fila IN
        SELECT lp.linea_id, p.nombre, lp.cantidad,
               round(lp.cantidad * lp.precio_unit, 2) AS subtotal
        FROM tienda.lineas_pedido AS lp
        JOIN tienda.productos AS p ON p.producto_id = lp.producto_id
        WHERE lp.pedido_id = p_pedido_id
        ORDER BY lp.linea_id
    LOOP
        v_linea_id := v_fila.linea_id;
        v_producto := v_fila.nombre;
        v_cantidad := v_fila.cantidad;
        v_subtotal := v_fila.subtotal;
        RETURN NEXT;
    END LOOP;
    RETURN;
END;
$$;

SELECT * FROM tienda.fn_lineas_pedido_implicito_record(1);
```

Resultado esperado: una fila para Teclado mecánico (`79.90`) y otra para Ratón inalámbrico (`79.00`).

8.3 Cursor explícito ligado (`bound cursor`) sin `OUT`

```
CREATE OR REPLACE FUNCTION tienda.fn_contar_productos_cursor_explicito(
    p_stock_minimo integer
)
RETURNS integer
LANGUAGE plpgsql
STABLE
AS $$
DECLARE
    v_cursor CURSOR (p_minimo integer) FOR
        SELECT producto_id FROM tienda.productos WHERE stock >= p_minimo;
    v_producto_id integer;
    v_contador integer := 0;
BEGIN
    OPEN v_cursor(p_stock_minimo);
    LOOP
        FETCH v_cursor INTO v_producto_id;
        EXIT WHEN NOT FOUND;
        v_contador := v_contador + 1;
    END LOOP;
    CLOSE v_cursor;
    RETURN v_contador;
END;
$$;

SELECT tienda.fn_contar_productos_cursor_explicito(50) AS v_total;
-- Resultado: 2
```

8.4 Cursor explícito con `RETURNS TABLE` y `record`

```
CREATE OR REPLACE FUNCTION tienda.fn_productos_cursor_explicito_record(
    p_stock_minimo integer
)
RETURNS TABLE(v_producto_id integer, v_nombre text, v_stock integer)
LANGUAGE plpgsql
STABLE
AS $$
```

```
DECLARE
    v_cursor refcursor;
    v_fila record;
BEGIN
    OPEN v_cursor FOR
        SELECT p.producto_id, p.nombre, p.stock
        FROM tienda.productos AS p
        WHERE p.stock >= p_stock_minimo
        ORDER BY p.producto_id;

    LOOP
        FETCH v_cursor INTO v_fila;
        EXIT WHEN NOT FOUND;
        v_producto_id := v_fila.producto_id;
        v_nombre := v_fila.nombre;
        v_stock := v_fila.stock;
        RETURN NEXT;
    END LOOP;
    CLOSE v_cursor;
    RETURN;
END;
$$;
```

8.5 Cursor explícito con `RETURNS TABLE`, sin `record`

```
CREATE OR REPLACE FUNCTION tienda.fn_productos_cursor_explicito_sin_record(
    p_stock_minimo integer
)
RETURNS TABLE(v_producto_id integer, v_nombre text, v_stock integer)
LANGUAGE plpgsql
STABLE
AS $$
DECLARE
    v_cursor refcursor;
BEGIN
    OPEN v_cursor FOR
        SELECT p.producto_id, p.nombre, p.stock
        FROM tienda.productos AS p
        WHERE p.stock >= p_stock_minimo
        ORDER BY p.producto_id;

    LOOP
        FETCH v_cursor INTO v_producto_id, v_nombre, v_stock;
        EXIT WHEN NOT FOUND;
        RETURN NEXT;
    END LOOP;
    CLOSE v_cursor;
    RETURN;
END;
$$;

SELECT * FROM tienda.fn_productos_cursor_explicito_sin_record(50);
```

La variante sin `record` es más explícita y PostgreSQL verifica que el número y tipos de columnas sean compatibles.

8.6 Devolver un cursor al llamador (`refcursor` con `OUT`)

Un cursor devuelto vive hasta el fin de la transacción. Por ello, `BEGIN`, llamada y `FETCH` deben estar en la misma transacción.

```
CREATE OR REPLACE FUNCTION tienda.fn_abrir_productos_cursor(
    IN p_stock_minimo integer,
    OUT v_cursor refcursor
)
LANGUAGE plpgsql
STABLE
AS $$
BEGIN
    v_cursor := 'v_cursor_productos';
    OPEN v_cursor FOR
        SELECT producto_id, nombre, precio, stock
        FROM tienda.productos
        WHERE stock >= p_stock_minimo
        ORDER BY producto_id;
END;
$$;
BEGIN;
```

```
SELECT tienda.fn_abrir_productos_cursor(20) AS v_cursor;
FETCH ALL FROM v_cursor_productos;
COMMIT;
```

Para consultas dinámicas de cursor: `OPEN v_cursor FOR EXECUTE v_sql USING p_valor;`. Para cambiar la fila corriente, abre con `FOR UPDATE` y usa `UPDATE ... WHERE CURRENT OF v_cursor`.

9. Procedimientos almacenados (`CREATE PROCEDURE`)

Enunciado

Crear un pedido, crear su línea y reducir stock de manera atómica.

```
CREATE OR REPLACE PROCEDURE tienda.sp_crear_linea_pedido(
    IN p_cliente_id integer,
    IN p_producto_id integer,
    IN p_cantidad integer,
    OUT v_pedido_id integer,
    OUT v_linea_id integer,
    OUT v_total_linea numeric(10,2)
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_precio_unit numeric(10,2);
    v_stock integer;
BEGIN
    IF p_cantidad <= 0 THEN
        RAISE EXCEPTION 'La cantidad debe ser mayor que cero';
    END IF;

    -- FOR UPDATE evita que dos ventas concurrentes descuenten el mismo stock.
    SELECT precio, stock
      INTO STRICT v_precio_unit, v_stock
      FROM tienda.productos
     WHERE producto_id = p_producto_id
       AND activo
     FOR UPDATE;

    IF v_stock < p_cantidad THEN
        RAISE EXCEPTION 'Stock insuficiente para producto %: disponible %, solicitado %',
            p_producto_id, v_stock, p_cantidad;
    END IF;

    INSERT INTO tienda.pedidos (cliente_id, estado)
    VALUES (p_cliente_id, 'pendiente')
    RETURNING pedido_id INTO v_pedido_id;

    INSERT INTO tienda.lineas_pedido (pedido_id, producto_id, cantidad, precio_unit)
    VALUES (v_pedido_id, p_producto_id, p_cantidad, v_precio_unit)
    RETURNING linea_id INTO v_linea_id;

    UPDATE tienda.productos
       SET stock = stock - p_cantidad
     WHERE producto_id = p_producto_id;

    v_total_linea := round(p_cantidad * v_precio_unit, 2);
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RAISE EXCEPTION 'Producto activo no encontrado: %', p_producto_id;
END;
$$;

-- Los lugares para OUT se pasan como NULL en CALL.
CALL tienda.sp_crear_linea_pedido(1, 1, 2, NULL, NULL, NULL);
```

Resultado esperado de `CALL`: una fila de salida con un nuevo `v_pedido_id`, el nuevo `v_linea_id` y `v_total_linea = 159.80`. Además, el stock del producto 1 disminuye en 2.

Procedimiento con `INOUT`

```
CREATE OR REPLACE PROCEDURE tienda.sp_normalizar_estado(
    INOUT v_estado text
)
LANGUAGE plpgsql
AS $$
BEGIN
    v_estado := lower(trim(v_estado));
    IF v_estado NOT IN ('pendiente', 'pagado', 'enviado', 'cancelado') THEN
        RAISE EXCEPTION 'Estado no permitido: %', v_estado;
    END IF;
END;
$$;

CALL tienda.sp_normalizar_estado(' PENDIENTE ');
-- Resultado: pendiente
```

No realices `COMMIT` o `ROLLBACK` de forma rutinaria dentro de procedimientos de negocio. Si necesitas control de transacciones, entiende primero las restricciones de `CALL`: no puede haber una cadena de comandos SQL que lo impida y no debe ejecutarse dentro de un bloque de transacción explícito para que el procedimiento pueda terminar/iniciar transacciones.

10. Agregaciones, subconsultas y consultas anidadas

Las funciones PL/pgSQL aprovechan SQL completo. En muchos casos, una sola consulta es preferible a un bucle.

Enunciado: total por cliente con subconsulta correlacionada

```
CREATE OR REPLACE FUNCTION tienda.fn_gasto_cliente(
    p_cliente_id integer
)
RETURNS numeric(12,2)
LANGUAGE plpgsql
STABLE
AS $$
DECLARE
    v_gasto numeric(12,2);
BEGIN
    SELECT COALESCE(SUM(
        (SELECT SUM(lp.cantidad * lp.precio_unit)
         FROM tienda.lineas_pedido AS lp
         WHERE lp.pedido_id = pe.pedido_id)
    ), 0)
    INTO v_gasto
    FROM tienda.pedidos AS pe
    WHERE pe.cliente_id = p_cliente_id;

    RETURN v_gasto;
END;
$$;
```

Alternativa normalmente más eficiente con `JOIN` y `GROUP BY`:

```
SELECT c.cliente_id, c.nombre,
       COALESCE(SUM(lp.cantidad * lp.precio_unit), 0) AS v_total
FROM tienda.clientes AS c
LEFT JOIN tienda.pedidos AS pe ON pe.cliente_id = c.cliente_id
LEFT JOIN tienda.lineas_pedido AS lp ON lp.pedido_id = pe.pedido_id
GROUP BY c.cliente_id, c.nombre;
```

Agregaciones indispensables:

```
SELECT
    COUNT(*) AS v_num_lineas,
    COUNT(DISTINCT producto_id) AS v_productos_distintos,
    SUM(cantidad) AS v_unidades,
    AVG(precio_unit) AS v_precio_medio,
    MIN(precio_unit) AS v_precio_minimo,
    MAX(precio_unit) AS v_precio_maximo,
    string_agg(producto_id::text, ', ' ORDER BY producto_id) AS v_productos
FROM tienda.lineas_pedido;
```

Recuerda: `SUM`, `AVG`, `MIN`, `MAX` sobre cero filas devuelven `NULL`; aplica `COALESCE(..., 0)` cuando el significado deseado sea cero.

11. Funciones de ventana

Las ventanas calculan sobre un conjunto de filas **sin colapsarlo**, a diferencia de `GROUP BY`.

Enunciado: clasificar empleados por salario dentro de su departamento

```
CREATE OR REPLACE FUNCTION tienda.fn_ranking_empleados()
RETURNS TABLE(
    v_empleado_id integer,
    v_nombre text,
    v_departamento text,
    v_salario numeric(10,2),
    v_rango_salario bigint,
    v_salario_acumulado numeric
)
LANGUAGE plpgsql
STABLE
AS $$
BEGIN
    RETURN QUERY
    SELECT e.id,
           e.nombre,
           e.departamento,
           e.salario,
           dense_rank() OVER (
               PARTITION BY e.departamento ORDER BY e.salario DESC
           ),
           sum(e.salario) OVER (
               PARTITION BY e.departamento
               ORDER BY e.salario DESC
               ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
           )
    FROM tienda.empleados AS e
    ORDER BY e.departamento, e.salario DESC;
END;
$$;

SELECT * FROM tienda.fn_ranking_empleados();
```

Funciones frecuentes: `row_number()` (número único), `rank()` (deja huecos ante empate), `dense_rank()` (no deja huecos), `lag()`/`lead()` (fila anterior/siguiente), `sum() OVER` (acumulado) y `avg() OVER` (media móvil). `PARTITION BY` reinicia el cálculo por grupo; `ORDER BY` define el orden lógico.

12. JSON y JSONB en PostgreSQL 18

- `json` conserva el texto JSON de entrada; `jsonb` lo descompone en formato binario, permite comparación e índices GIN y suele ser la opción para consultar/manipular.
- `json_agg`/`jsonb_agg` convierten filas/valores en arrays.
- `json_build_object`/`jsonb_build_object` construyen objetos con pares clave-valor.
- `to_json(anyelement)` y `to_jsonb(anyelement)` convierten cualquier valor SQL, arrays o tipos compuestos recursivamente.
- `row_to_json(record)` construye un objeto desde una fila.
- `json_serialize` genera texto o `bytea` desde una expresión SQL/JSON.

12.1 Pedido con líneas: `json_agg` + `json_build_object`

```
CREATE OR REPLACE FUNCTION tienda.fn_pedido_json(
    p_pedido_id integer
)
RETURNS json
LANGUAGE plpgsql
STABLE
AS $$
DECLARE
    v_documento json;
BEGIN
    SELECT json_build_object(
        'pedido_id', pe.pedido_id,
        'cliente', c.nombre,
        'estado', pe.estado,
        'lineas', COALESCE(
            json_agg(
                json_build_object(
                    'linea_id', lp.linea_id,
                    'producto', pr.nombre,
                    'cantidad', lp.cantidad,
                    'precio_unit', lp.precio_unit,
                    'subtotal', lp.cantidad * lp.precio_unit
                ) ORDER BY lp.linea_id
            ) FILTER (WHERE lp.linea_id IS NOT NULL),
            '[]'::json
        )
    )
    INTO STRICT v_documento
    FROM tienda.pedidos AS pe
    JOIN tienda.clientes AS c ON c.cliente_id = pe.cliente_id
    LEFT JOIN tienda.lineas_pedido AS lp ON lp.pedido_id = pe.pedido_id
    LEFT JOIN tienda.productos AS pr ON pr.producto_id = lp.producto_id
    WHERE pe.pedido_id = p_pedido_id
    GROUP BY pe.pedido_id, c.nombre, pe.estado;

    RETURN v_documento;
END;
$$;

SELECT json_pretty(tienda.fn_pedido_json(1));
```

Resultado esperado (resumido)

```
{
  "pedido_id": 1,
  "cliente": "Juan Pérez",
  "estado": "pendiente",
  "lineas": [
    {"linea_id": 1, "producto": "Teclado mecánico", "cantidad": 1, "precio_unit": 79.90, "subtotal": 79.90},
    {"linea_id": 2, "producto": "Ratón inalámbrico", "cantidad": 2, "precio_unit": 39.50, "subtotal": 79.00}
  ]
}
```

`FILTER (WHERE ...)` evita que el `LEFT JOIN` agregue un objeto ficticio de valores nulos. `ORDER BY` dentro de `json_agg` hace el array determinista.

12.2 `jsonb_agg` + `jsonb_build_object` y operadores JSONB

```
SELECT jsonb_agg(
           jsonb_build_object(
               'producto_id', p.producto_id,
               'nombre', p.nombre,
               'stock', p.stock
           ) ORDER BY p.producto_id
       ) AS v_productos_jsonb
FROM tienda.productos AS p
WHERE p.activo;

SELECT ('{"cliente":{"nombre":"Juan"},"estado":"pendiente"}'::jsonb
        #>> '{cliente,nombre}') AS v_nombre;
-- Resultado: Juan
```

Operadores habituales: `->` devuelve JSON/JSONB, `->>` devuelve `text`, `#>` navega una ruta y `@>` comprueba contención en `jsonb`.

12.3 `to_json`, `to_jsonb` y `row_to_json`

```
SELECT to_json(ARRAY[1, 2, 3]) AS v_array_json,
       to_jsonb(ROW(1, 'Teclado', 79.90)) AS v_fila_jsonb;

SELECT row_to_json(p) AS v_producto_json
FROM (
    SELECT producto_id, nombre, precio, stock
    FROM tienda.productos
    WHERE producto_id = 1
) AS p;
```

`to_json(anyelement)` y `to_jsonb(anyelement)` sirven para escalares, arrays y compuestos. `row_to_json` es particularmente claro cuando partes de una fila o subconsulta con columnas bien nombradas.

12.4 `json_serialize` en PostgreSQL 18

```
SELECT json_serialize(
           to_jsonb(ROW(1, 'Teclado mecánico', 79.90))
           RETURNING text
       ) AS v_texto_json;

SELECT json_serialize(
           '{"pedido_id": 1}'::json
           RETURNING bytea
       ) AS v_bytes_json;
```

`json_serialize` forma parte de SQL/JSON y convierte una expresión JSON a `text` (predeterminado) o `bytea`; no debe confundirse con `to_jsonb`, que convierte valores SQL *a* JSONB.

12.5 Retornar JSONB desde PL/pgSQL

```
CREATE OR REPLACE FUNCTION tienda.fn_producto_jsonb(
    p_producto_id integer
)
RETURNS jsonb
LANGUAGE plpgsql
STABLE
AS $$
DECLARE
    v_producto_jsonb jsonb;
BEGIN
    SELECT to_jsonb(p)
      INTO STRICT v_producto_jsonb
      FROM tienda.productos AS p
     WHERE p.producto_id = p_producto_id;
    RETURN v_producto_jsonb;
END;
$$;
```

Para JSONB consultado frecuentemente, considera un índice: `CREATE INDEX ... ON tabla USING gin (columna_jsonb);`. No crees índices sin validar el patrón real de consulta y escritura.

13. Errores, validación y buenas prácticas

```
CREATE OR REPLACE FUNCTION tienda.fn_cambiar_estado(
    p_pedido_id integer,
    p_estado_nuevo text
)
RETURNS void
LANGUAGE plpgsql
AS $$
DECLARE
    v_estado_actual text;
BEGIN
    SELECT estado INTO STRICT v_estado_actual
    FROM tienda.pedidos
    WHERE pedido_id = p_pedido_id
```

```
    FOR UPDATE;

    IF p_estado_nuevo NOT IN ('pendiente', 'pagado', 'enviado', 'cancelado') THEN
        RAISE EXCEPTION 'Estado inválido: %', p_estado_nuevo
            USING ERRCODE = '22023', HINT = 'Use pendiente, pagado, enviado o cancelado';
    END IF;

    IF v_estado_actual = 'cancelado' THEN
        RAISE EXCEPTION 'Un pedido cancelado no puede cambiar de estado';
    END IF;

    UPDATE tienda.pedidos
       SET estado = p_estado_nuevo
     WHERE pedido_id = p_pedido_id;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RAISE EXCEPTION 'Pedido inexistente: %', p_pedido_id;
END;
$$;
```

Recomendaciones profesionales:

1. Deja las reglas de integridad fundamentales en la base: `NOT NULL`, claves foráneas, `UNIQUE`, `CHECK` y transacciones; el código PL/pgSQL las complementa, no las sustituye.
2. Usa `STRICT` cuando la lógica requiera una sola fila. Usa `COALESCE` para definir el comportamiento de agregados vacíos.
3. En modificaciones concurrentes, bloquea la fila necesaria con `FOR UPDATE` o diseña un `UPDATE ... WHERE stock >= p_cantidad RETURNING ...` atómico.
4. Evita SQL dinámico si SQL estático resuelve el caso; si es imprescindible, usa `format('%I', ...)` para identificadores y `USING` para valores.
5. Expón firmas claras con `RETURNS TABLE` en vez de `SETOF record` cuando conozcas las columnas.
6. Documenta las funciones con `COMMENT ON FUNCTION`, prueba entradas válidas, límites, inexistentes y concurrencia.

Seguridad de rutinas

Por defecto se ejecutan como `SECURITY INVOKER`: respetan los permisos de quien llama. `SECURITY DEFINER` permite ejecutarlas con los permisos de su propietario, y exige especial cuidado: fija un `search_path` seguro, cualifica los objetos y no construyas SQL con valores concatenados.

```
CREATE OR REPLACE FUNCTION tienda.fn_total_pedidos_seguro()
RETURNS bigint
LANGUAGE sql
SECURITY DEFINER
SET search_path = tienda, pg_catalog
AS $$
    SELECT count(*) FROM tienda.pedidos;
$$;

REVOKE ALL ON FUNCTION tienda.fn_total_pedidos_seguro() FROM PUBLIC;
-- Ejecuta solo después de crear el rol correspondiente:
-- GRANT EXECUTE ON FUNCTION tienda.fn_total_pedidos_seguro() TO rol_aplicacion;
```

Sustituye `rol_aplicacion` por un rol real. La cualificación `tienda.pedidos` y `SET search_path` evitan que un objeto malicioso situado antes en la ruta cambie qué tabla lee la rutina.

14. Mapa de decisión rápido

Necesidad	Declaración recomendada
Un dato	`RETURNS tipo` + `RETURN v_valor`
Una fila de salida con nombres	parámetros `OUT`
Muchas filas y columnas conocidas	`RETURNS TABLE(v_...)` + `RETURN QUERY`

Necesidad	Declaración recomendada
Muchas filas de una tabla	`RETURNS SETOF tienda.tabla`
Forma de salida variable	`RETURNS SETOF record` + definición de columnas en llamada
Efecto de negocio por `CALL`	`CREATE PROCEDURE`, con `OUT`/`INOUT` si hace falta
Recorrer filas sin control manual	`FOR v_fila IN SELECT` (cursor implícito)
Leer por bloques/devolver portal	`refcursor`, `OPEN`, `FETCH`, `CLOSE`
Tabla/columna dinámica	`EXECUTE format(...) USING ...`
Documento para API	`jsonb_build_object` + `jsonb_agg`

15. Consultas de comprobación final

```
SELECT * FROM tienda.fn_resumen_pedido_out(1);
SELECT * FROM tienda.fn_productos_por_stock(20);
SELECT * FROM tienda.fn_productos_record(20)
    AS r(v_producto_id integer, v_nombre text, v_stock integer);
SELECT * FROM tienda.fn_ranking_empleados();
SELECT json_pretty(tienda.fn_pedido_json(1));
SELECT tienda.fn_stock_producto(1);
```

Para profundizar, consulta la documentación oficial de PostgreSQL 18 sobre PL/pgSQL, cursores y funciones JSON.

16. Anexo: comparativa rápida de cursores

Combinación	Declaración	Apertura	Extracción	Devolución
Implícito + sin OUT	`FOR v_registro IN ...`	Automática	Automática	`RETURN NEXT v_registro;`
Implícito + con OUT	`FOR v_registro IN ...`	Automática	Automática	Asignar OUT + `RETURN NEXT;`
Explícito + sin OUT	`v_cursor CURSOR FOR ...`	`OPEN`	`FETCH ... INTO v_fila`	`RETURN NEXT v_fila;`
Explícito + con OUT	`v_cursor CURSOR FOR ...`	`OPEN`	`FETCH ... INTO v_fila`	Asignar OUT + `RETURN NEXT;`
Explícito + escalares	`v_cursor CURSOR FOR ...`	`OPEN`	`FETCH ... INTO v_a, v_b`	`RETURN NEXT ROW(v_a, v_b, ...);` o asignar OUT + `RETURN NEXT;`

> *En PL/pgSQL,* `RETURN NEXT` *recibe una única expresión. Por eso no se escribe* `RETURN NEXT v_a, v_b`*; para varios valores se devuelve una fila compuesta con* `ROW(...)` *o se usan variables de salida* `OUT`*.*

17. Anexo: reglas de oro con `SETOF record`

1. Si la firma no declara columnas `OUT`, el llamador debe declarar la estructura con `AS t(...)`. Sin ello PostgreSQL no sabe qué columnas esperar y lanza un error.
2. Los nombres, el orden y los tipos de `AS t(...)` deben ser compatibles con lo que devuelve la función. El error se detecta en tiempo de ejecución.
3. Prefiere `RETURNS TABLE` cuando la estructura sea fija: aporta una interfaz tipada y evita que el llamador repita `AS t(...)`.
4. Usa `SETOF record` cuando la estructura sea variable o desconocida, por ejemplo en SQL dinámico, informes parametrizables o funciones genéricas.
5. Con cursores explícitos, ejecuta siempre `CLOSE v_cursor` para liberar recursos. Para garantía adicional, ciérralo también en el bloque de excepciones cuando la función pueda fallar dentro del bucle.
6. Todas las variables declaradas siguen el prefijo `v_` del tutorial: `v_registro`, `v_producto`, `v_cursor` y `v_cliente_id`.

> *Excepción importante: una función que declara varios* `OUT` *y* `RETURNS SETOF record` *ya define sus columnas de salida; en ese caso se puede invocar directamente con* `SELECT * FROM funcion(...);`.